AUTOMATED DISA STIG COMPLIANCE FOR CLOUD INFRASTRUTURE USING ANISBLE

Provisioning and Hardening Two Digital Ocean Droplets with DISA STIGs Using Ansible Automation Tool

CHARLES TULL

COURSE: PROJECT & PORTFOLIO 7

TABLE OF CONTENTS

PROJECT SCOPE OVERVIEW 2

NETWORK TOPOLOGY DIAGRAM WITH IP ADDRESSES 3

CLOUD AUTOMATION PROCEDURAL INSTRUCTIONS 4

- Milestone 1 - Build the Control Node and Prepare Credentials 4
- Milestone 2 - Provision the Droplets and Establish SSH Trust 6
- Milestone 3 - Apply and Validate STIG Hardening 10

FINAL SCRIPT 17

LESSONS LEARNED 24

APPENDIX A | DOCUMENT ERRORS & SOLUTIONS 26

APPENDIX B | DISA STIGS LIST 28

Scope of Work

PROJECT SCOPE OVERVIEW

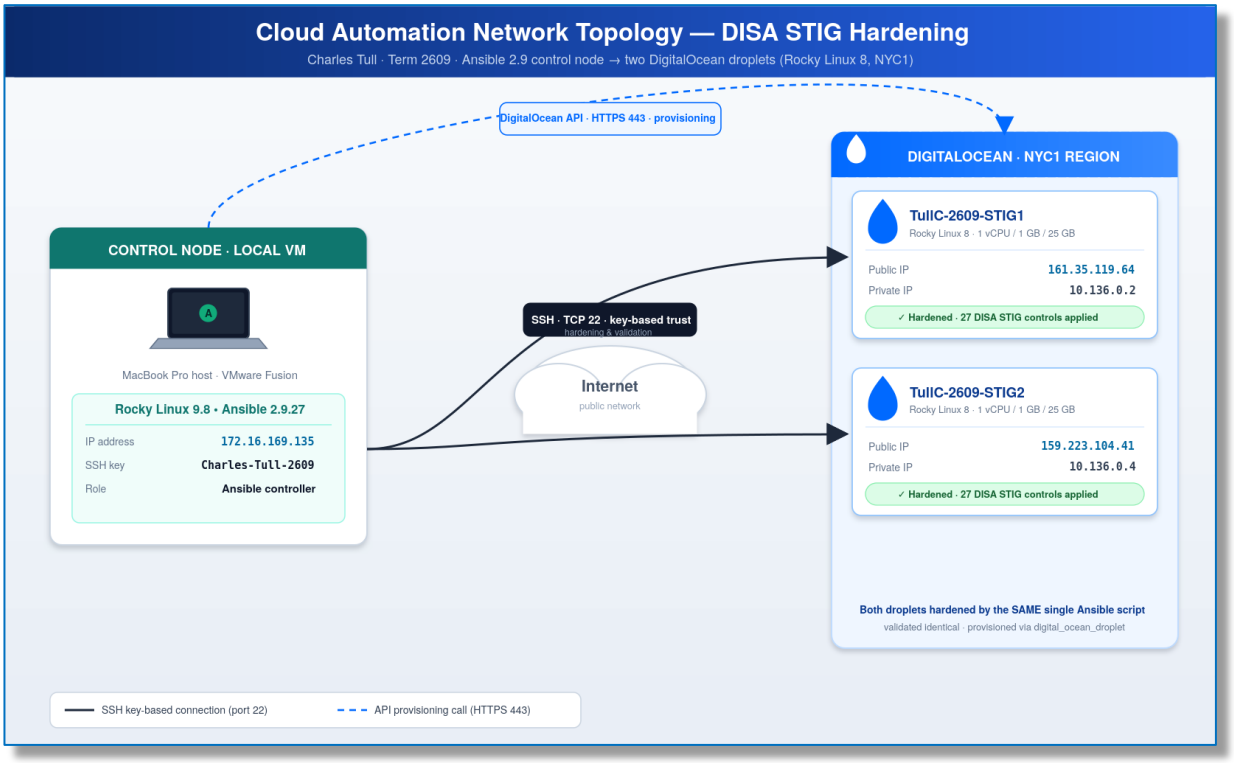
Project Scope:

This project rebuilds and secures the cloud infrastructure I created in Project & Portfolio VI. The purpose is to use Ansible automation and SSH trust to fully provision two DigitalOcean droplets and then harden them by applying DISA STIG best practices, completing the entire process with a single Ansible script and as much automation as possible. This addresses the real-world need to stand up cloud servers that are secure and consistently configured from the moment they are created, rather than hardened by hand afterward.

Specifically, the project delivers three things: (1) a Rocky Linux control node running Ansible 2.9 that drives all of the automation; (2) two identical Rocky Linux 8 droplets (1 vCPU, 1 GB RAM, 25 GB disk) provisioned in the DigitalOcean NYC1 region and named TullC-2609-STIG1 and TullC-2609-STIG2; and (3) 27 DISA STIG (RHEL 8) controls applied to both droplets, followed by a validation run that confirms the controls are in effect and identical across both machines.

Success is measurable: 27 STIG controls (within the required 20-30 range) applied by one script, both droplets completing with failed=0, and a validation run reporting identical PASS results on both hosts. The scope is attainable and realistic because it builds directly on the Ansible and DigitalOcean environment I set up in PP6 and uses small, low-cost droplets. The project is time-bound, to be completed within the Project & Portfolio VII course term (Term 2609).

NETWORK TOPOLOGY DIAGRAM WITH IP ADDRESSES



CLOUD AUTOMATION PROCEDURAL INSTRUCTIONS

This section walks through the entire project from start to finish as step-by-step instructions. I organized the work into three milestones: Milestone 1 covers building the Ansible control node and preparing my credentials, Milestone 2 covers provisioning the two droplets and proving SSH trust, and Milestone 3 covers applying and validating the DISA STIG hardening. The Ansible scripts I wrote are included at the step where each one is created.

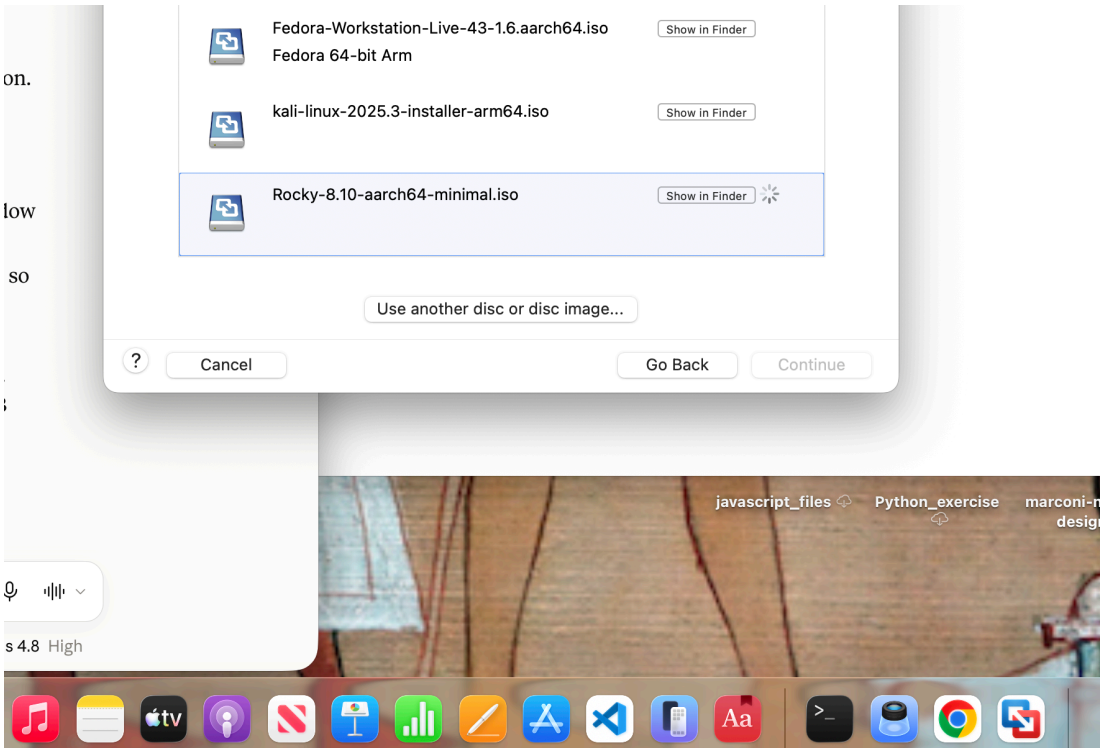
MILESTONE 1 - BUILD THE CONTROL NODE AND PREPARE CREDENTIALS

Step 1: Download the Rocky Linux ISO

I went to rockylinux.org/download to get the installer image for my control node. Because my Mac is Apple Silicon, I chose the ARM (aarch64) architecture and downloaded the Rocky Linux 8 Minimal ISO. On an Intel Mac I would have picked the x86_64 image instead.

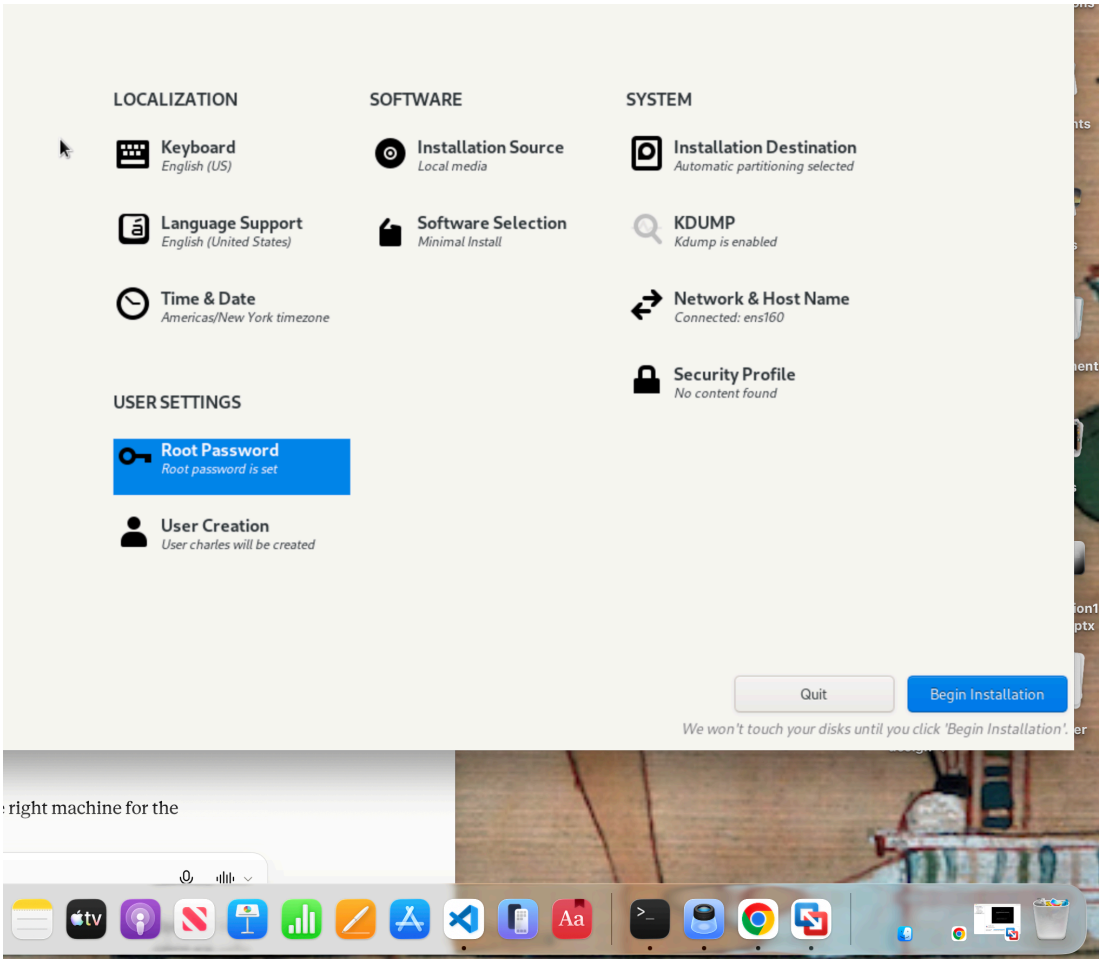
Step 2: Create the virtual machine in VMware Fusion

I opened VMware Fusion and chose File > New to start the Create a New Virtual Machine wizard. I selected "Install from disc or image," pointed it at the Rocky ISO I just downloaded, and clicked Continue. On the finish screen I clicked Customize Settings (instead of just Finish) so I could set the resources first: 2 CPUs, 2 GB of RAM, and a 20 GB disk. These are for the control node, which only runs Ansible, so it does not need to match the 1 CPU / 1 GB droplet specification. I named the VM rocky-control and saved it, then clicked the Play button to boot the installer.



Step 3: Install Rocky Linux

The Anaconda installer opened. I selected English and clicked Continue to reach the Installation Summary screen. From there I: (a) opened Installation Destination and clicked Done to accept the default automatic partitioning; (b) opened Network & Host Name and toggled the network On so the machine could reach the internet; (c) set a Root Password; and (d) under User Creation, created my user "charles." I then clicked Begin Installation, waited for it to finish, and rebooted into the new system.



Step 4: Connect to the VM from my Mac over SSH

To work more comfortably (and to get copy-and-paste), I connected to the VM from my Mac terminal instead of typing in the VMware console. First, inside the VM, I found its IP address:

```
ip addr show
```

I looked for the inet address under the ens160 interface. Then, from a terminal on my Mac, I connected to it:

```
ssh charles@<VM-IP-address>
```

Step 5: Install Ansible 2.9

On the control node I installed pip and git, then created an isolated Python virtual environment and pinned Ansible to exactly version 2.9. I also pinned Jinja2 below 3.1, because Ansible 2.9 uses functions that newer Jinja2 removed:

```
sudo dnf install -y python3-pip git

python3 -m venv ~/ansible29

source ~/ansible29/bin/activate

pip install --upgrade pip

pip install "ansible==2.9.27" "jinja2<3.1"
```

I confirmed the version, which reported ansible 2.9.27. I have to run the "source" command in every new terminal session to re-activate this environment before using Ansible.

```
ansible --version
```



Step 6: Generate the SSH key pair

I created a dedicated SSH key pair that Ansible uses to reach the droplets without a password. I used an empty passphrase (pressing Enter twice) so the automation can run unattended:

```
ssh-keygen -t ed25519 -C "Charles-Tull-2609" -f ~/.ssh/Charles-Tull-2609
```

This created the private key (which stays on the control node) and the public key, Charles-Tull-2609.pub, which I displayed so I could copy it into DigitalOcean:

```
cat ~/.ssh/Charles-Tull-2609.pub
```

Step 7: Add the SSH key and create an API token in DigitalOcean

In the DigitalOcean web console I went to Settings > Security > Add SSH Key, pasted the public key, and named it Charles-Tull-2609 (the name matters because the playbook looks the key up by name). Next I went to the API section and generated a Personal Access Token with full read/write access, which Ansible needs to create droplets. I exported the token as an environment variable on the control node so it never has to be written into a file:

```
export DO_API_TOKEN="<my token>"
```

MILESTONE 2 - PROVISION THE DROPLETS AND ESTABLISH SSH TRUST

Step 8: Create the project folder and Ansible configuration

I made a project directory and moved into it. Everything for the project lives here:

```
mkdir -p ~/stig-project

cd ~/stig-project
```

Then I created ansible.cfg, which sets defaults (like the SSH key and disabling host-key checking) so Ansible can run against brand-new droplets without stopping to ask questions:

ansible.cfg

```
[defaults]
# Fresh droplets present new SSH host keys - disabling this stops Ansible
# from stopping to ask "trust this host?" so it can run unattended.
host_key_checking = False
# The private key whose public half is attached to both droplets.
private_key_file = ~/.ssh/Charles-Tull-2609
# Connect to the droplets as root by default.
remote_user = root
# Use this inventory file unless told otherwise.
inventory = ./inventory.ini
retry_files_enabled = False
```

Step 9: Write the provisioning playbook

This is the script that creates both droplets. It reads my API token from the environment, looks up my SSH key by name, and creates two identical Rocky Linux 8 droplets sized to the project spec (1 vCPU, 1 GB RAM, 25 GB disk) in the NYC1 region. The unique_name setting makes it safe to run more than once. I saved this as 01-provision.yml:

01-provision.yml

```
---
- name: Provision two DigitalOcean droplets for STIG hardening
  hosts: localhost          # runs on the control node itself
  connection: local         # calls the DO API locally, not over SSH
  gather_facts: false
  vars:
    # Read the API token from the environment, never hard-code it.
    do_token: "{{ lookup('env', 'DO_API_TOKEN') }}"
```

```
  ssh_key_name: "Charles-Tull-2609"     # must match the key name in DO
  droplet_region: "nyc1"
  droplet_size: "s-1vcpu-1gb"           # 1 vCPU / 1 GB / 25 GB - meets spec
  droplet_image: "rockylinux-8-x64"     # Rocky 8 -> DISA RHEL 8 STIG
  droplet_names:
    - "TullC-2609-STIG1"
    - "TullC-2609-STIG2"
tasks:
  - name: Look up SSH keys on the DO account
    digital_ocean_sshkey_info:
      oauth_token: "{{ do_token }}"
    register: do_keys

  - name: Extract the ID of our named key
    set_fact:
      ssh_key_id: "{{ (do_keys.data | selectattr('name','equalto', ssh_key_name) | list | first).id }}"

  - name: Create the two droplets
    digital_ocean_droplet:
      state: present
      oauth_token: "{{ do_token }}"
      name: "{{ item }}"
      unique_name: true                 # idempotent - safe to re-run
      region: "{{ droplet_region }}"
      size: "{{ droplet_size }}"
      image: "{{ droplet_image }}"
      ssh_keys:
        - "{{ ssh_key_id }}"
    loop: "{{ droplet_names }}"
    register: droplets

  - name: Show the assigned public IPs
    debug:
      msg: "{{ item.data.droplet.name }} -> {{ item.data.ip_address | default('IP pending...') }}"
    loop: "{{ droplets.results }}"
    loop_control:
```

```
label: "{{ item.item }}"
```

Step 10: Run the provisioning playbook

With my token exported, I ran the playbook. It took about a minute while DigitalOcean built the two machines, then printed each droplet's public IP address. A clean run ends with a PLAY RECAP showing failed=0:

```
ansible-playbook 01-provision.yml
```

```
(ansible29) [charles@localhost stig-project]$ ansible-playbook provision.yml
[WARNING]: Unable to parse /home/charles/stig-project/inventory.ini as an inventory source
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit
localhost does not match 'all'

PLAY [Provision two DigitalOcean droplets for STIG hardening] ********************************

TASK [Look up SSH keys on the DO account] ****************************************************
ok: [localhost]

TASK [Extract the ID of our named key] *******************************************************
ok: [localhost]

TASK [Create the two droplets] ***************************************************************
changed: [localhost] => (item=TullC-2609-STIG1)
changed: [localhost] => (item=TullC-2609-STIG2)

TASK [Show the assigned public IPs] **********************************************************
ok: [localhost] => (item=TullC-2609-STIG1) => {
    "msg": "TullC-2609-STIG1 -> 161.35.119.64"
}
ok: [localhost] => (item=TullC-2609-STIG2) => {
    "msg": "TullC-2609-STIG2 -> 159.223.104.41"
}

PLAY RECAP ***********************************************************************************
localhost                  : ok=4    changed=1    unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0

(ansible29) [charles@localhost stig-project]$
```

Step 11: Confirm the droplets in the DigitalOcean console

I verified in the web console that both droplets (TullC-2609-STIG1 and TullC-2609-STIG2) were Active and running Rocky Linux 8 in the NYC1 region, exactly matching the specification.



Step 12: Build the inventory and verify SSH trust

I created inventory.ini, which tells Ansible how to reach the two droplets by their IP addresses and which SSH key to use:

inventory.ini

```
[stig_targets]

# Each line: <droplet name> and its public IP (from the provisioning run).

TullC-2609-STIG1 ansible_host=161.35.119.64

TullC-2609-STIG2 ansible_host=159.223.104.41


[stig_targets:vars]

ansible_user=root

ansible_ssh_private_key_file=~/.ssh/Charles-Tull-2609
```

Then I tested the connection to both droplets with an Ansible ping. Getting "pong" back from both machines proves the SSH-trust chain works end to end - my control node reaches each droplet using the key with no password. This is what every hardening task depends on:

```
ansible stig_targets -m ping

PLAY RECAP *********************************************************************************
localhost                  : ok=4    changed=1    unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0

(ansible29) [charles@localhost stig-project]$ cat > ~/stig-project/inventory.ini << 'EOF'
[stig_targets]
TullC-2609-STIG1 ansible_host=161.35.119.64
TullC-2609-STIG2 ansible_host=159.223.104.41

[stig_targets:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/Charles-Tull-2609
EOF
(ansible29) [charles@localhost stig-project]$ ansible stig_targets -m ping
TullC-2609-STIG2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/libexec/platform-python"
    },
    "changed": false,
    "ping": "pong"
}
TullC-2609-STIG1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/libexec/platform-python"
    },
    "changed": false,
    "ping": "pong"
}
(ansible29) [charles@localhost stig-project]$
```

MILESTONE 3 - APPLY AND VALIDATE STIG HARDENING

Step 13: Write the hardening playbook

This is the single hardening script - the heart of the project. It applies 27 DISA STIG (RHEL 8) controls, grouped by area: SSH hardening, password and authentication policy, auditing, kernel and network

settings, file permissions, and removing insecure services. Every task enforces a desired state rather than blindly running a command, which is what makes the results identical and repeatable across both machines. I saved this as 02-harden.yml:

02-harden.yml

```
---

- name: Apply DISA STIG hardening to both droplets

  hosts: stig_targets

  become: true

  handlers:

    - name: restart sshd

      service: { name: sshd, state: restarted }

  tasks:

    # ===== GROUP 1 - SSH HARDENING =====

    - name: RHEL-08-010550 - Disable SSH root password login

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?PermitRootLogin', line: 'PermitRootLogin
prohibit-password' }

      notify: restart sshd

    - name: RHEL-08-010830 - Disable SSH empty passwords

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?PermitEmptyPasswords', line:
'PermitEmptyPasswords no' }

      notify: restart sshd

    - name: RHEL-08-010200 - Set SSH idle timeout (600s)

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?ClientAliveInterval', line:
'ClientAliveInterval 600' }

      notify: restart sshd

    - name: RHEL-08-010201 - Set SSH ClientAliveCountMax to 1

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?ClientAliveCountMax', line:
'ClientAliveCountMax 1' }

      notify: restart sshd

    - name: RHEL-08-020022 - Disable SSH X11 forwarding

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?X11Forwarding', line: 'X11Forwarding no' }

      notify: restart sshd

    - name: RHEL-08-010522 - Disable SSH PermitUserEnvironment

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?PermitUserEnvironment', line:
'PermitUserEnvironment no' }

      notify: restart sshd

    # ===== GROUP 2 - PASSWORD & AUTHENTICATION POLICY =====

    - name: RHEL-08-020230 - Minimum password length 15

      lineinfile: { path: /etc/security/pwquality.conf, regexp: '^#?minlen', line: 'minlen = 15' }
```

```
    - name: RHEL-08-0201xx - Require all 4 character classes

      lineinfile: { path: /etc/security/pwquality.conf, regexp: "^#?{{ item.key }}", line: "{{ item.key }}
= {{ item.value }}" }

      loop:

        - { key: 'ucredit', value: '-1' }

        - { key: 'lcredit', value: '-1' }

        - { key: 'dcredit', value: '-1' }

        - { key: 'ocredit', value: '-1' }

    - name: RHEL-08-020200 - Maximum password age 60 days

      lineinfile: { path: /etc/login.defs, regexp: '^PASS_MAX_DAYS', line: 'PASS_MAX_DAYS 60' }

    - name: RHEL-08-020190 - Minimum password age 1 day

      lineinfile: { path: /etc/login.defs, regexp: '^PASS_MIN_DAYS', line: 'PASS_MIN_DAYS 1' }

    - name: RHEL-08-020221 - Prohibit reuse of last 5 passwords

      lineinfile: { path: /etc/security/pwquality.conf, regexp: '^#?remember', line: 'remember = 5' }

    # ===== GROUP 3 - AUDITING (auditd) =====

    - name: RHEL-08-030180 - Ensure auditd is installed

      package: { name: audit, state: present }

    - name: RHEL-08-030181 - Ensure auditd is enabled and running

      service: { name: auditd, state: started, enabled: true }

    - name: RHEL-08-030610 - Restrict audit log permissions

      file: { path: /var/log/audit/audit.log, mode: '0600', state: file }

      ignore_errors: true

    - name: RHEL-08-030550 - Audit privileged (sudo) commands

      lineinfile: { path: /etc/audit/rules.d/stig.rules, line: '-w /usr/bin/sudo -p x -k privileged',
create: true }

    # ===== GROUP 4 - KERNEL & NETWORK (sysctl) =====

    - name: RHEL-08-040259 - Disable ICMP redirect acceptance

      sysctl: { name: net.ipv4.conf.all.accept_redirects, value: '0', state: present, sysctl_set: true,
reload: true }

    - name: RHEL-08-040249 - Disable source-routed packets

      sysctl: { name: net.ipv4.conf.all.accept_source_route, value: '0', state: present, sysctl_set: true,
reload: true }

    - name: RHEL-08-040239 - Enable reverse path filtering

      sysctl: { name: net.ipv4.conf.all.rp_filter, value: '1', state: present, sysctl_set: true, reload:
true }

    - name: RHEL-08-040281 - Log martian packets

      sysctl: { name: net.ipv4.conf.all.log_martians, value: '1', state: present, sysctl_set: true,
reload: true }

    - name: RHEL-08-010375 - Enable kernel ASLR
```

```
      sysctl: { name: kernel.randomize_va_space, value: '2', state: present, sysctl_set: true, reload:
true }

    # ===== GROUP 5 - FILE PERMISSIONS & SYSTEM INTEGRITY =====

    - name: RHEL-08-010340 - /etc/passwd permissions 0644

      file: { path: /etc/passwd, owner: root, group: root, mode: '0644' }

    - name: RHEL-08-010030 - /etc/shadow permissions 0000

      file: { path: /etc/shadow, owner: root, group: root, mode: '0000' }

    - name: RHEL-08-010350 - /etc/gshadow permissions 0000

      file: { path: /etc/gshadow, owner: root, group: root, mode: '0000' }

    - name: RHEL-08-010060 - Configure login warning banner

      copy:

        dest: /etc/issue

        content: "You are accessing a U.S. Government (USG) Information System (IS) that is provided for
USG-authorized use only.\n"

        owner: root

        group: root

        mode: '0644'

    - name: RHEL-08-040170 - Disable Ctrl-Alt-Del burst action

      systemd: { name: ctrl-alt-del.target, masked: true }

    # ===== GROUP 6 - INSECURE SERVICES =====

    - name: RHEL-08-040000 - Ensure telnet-server is removed

      package: { name: telnet-server, state: absent }
```

Step 14: Run the hardening playbook and confirm idempotency

I ran the hardening script against both droplets at once. Because the same script runs on both, they end up in the same state - the PLAY RECAP showed identical results on each host (ok=28, changed=20, failed=0). Running the script a second time reported changed=0, which proves it is idempotent and that both machines are in the same hardened configuration:

```
ansible-playbook 02-harden.yml
```

```
TASK [STIG RHEL-08-010030 - Set /etc/shadow permissions to 0000] ****************************
ok: [TullC-2609-STIG2]
ok: [TullC-2609-STIG1]

TASK [STIG RHEL-08-010350 - Set /etc/gshadow permissions to 0000] ***************************
ok: [TullC-2609-STIG2]
ok: [TullC-2609-STIG1]

TASK [STIG RHEL-08-010060 - Configure a login warning banner] ********************************
changed: [TullC-2609-STIG2]
changed: [TullC-2609-STIG1]

TASK [STIG RHEL-08-040170 - Disable Ctrl-Alt-Del burst action] *******************************
changed: [TullC-2609-STIG2]
changed: [TullC-2609-STIG1]

TASK [STIG RHEL-08-040000 - Ensure insecure telnet-server is removed] ************************
ok: [TullC-2609-STIG2]
ok: [TullC-2609-STIG1]

RUNNING HANDLER [restart sshd] ***************************************************************
changed: [TullC-2609-STIG2]
changed: [TullC-2609-STIG1]

PLAY RECAP ***********************************************************************************
TullC-2609-STIG1           : ok=28   changed=20   unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0
TullC-2609-STIG2           : ok=28   changed=20   unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0

(ansible29) [charles@localhost stig-project]$
```

Step 15: Write the validation playbook

To prove the hardening actually took effect (rather than just trusting the recap), I wrote a separate validation script. It reads the live configuration of key controls on both droplets and reports PASS or FAIL for each. I saved this as 03-validate.yml:

03-validate.yml

```
---
- name: Validate DISA STIG compliance on both droplets
  hosts: stig_targets
  become: true
  gather_facts: false
  tasks:
    - name: Read effective sshd configuration
      command: sshd -T
      register: sshd_config
      changed_when: false
    - name: Report SSH hardening status
      debug:
        msg: >-
```

```
          PermitRootLogin: {{ 'PASS' if 'permitrootlogin prohibit-password' in sshd_config.stdout else
'FAIL' }} |
          X11Forwarding: {{ 'PASS' if 'x11forwarding no' in sshd_config.stdout else 'FAIL' }}


    - name: Read pwquality minlen
      command: grep '^minlen' /etc/security/pwquality.conf
      register: minlen_check
      changed_when: false
      failed_when: false
    - name: Report password length status
      debug:
        msg: "Password minlen >= 15: {{ 'PASS' if '15' in minlen_check.stdout else 'FAIL' }}"


    - name: Check auditd is active
      command: systemctl is-active auditd
      register: auditd_check
      changed_when: false
      failed_when: false
    - name: Report auditd status
      debug:
        msg: "auditd running: {{ 'PASS' if auditd_check.stdout == 'active' else 'FAIL' }}"


    - name: Read /etc/shadow permissions
      stat: { path: /etc/shadow }
      register: shadow_stat
    - name: Report /etc/shadow permission status
      debug:
        msg: "/etc/shadow mode 0000: {{ 'PASS' if shadow_stat.stat.mode == '0000' else 'FAIL' }}"


    - name: Check telnet-server is not installed
      command: rpm -q telnet-server
      register: telnet_check
      changed_when: false
      failed_when: false
    - name: Report telnet removal status
      debug:
        msg: "telnet-server removed: {{ 'PASS' if 'not installed' in telnet_check.stdout else 'FAIL' }}"
```

Step 16: Run the validation playbook

Finally, I ran the validation script. Every control reported PASS, and - most importantly - the results were identical on both droplets, which is the concrete proof that the outcomes are the same on both machines:

```
ansible-playbook 03-validate.yml
```

```
TASK [Report /etc/shadow permission status] ************************************************
ok: [TullC-2609-STIG1] => {
    "msg": "/etc/shadow mode 0000: PASS"
}
ok: [TullC-2609-STIG2] => {
    "msg": "/etc/shadow mode 0000: PASS"
}

TASK [Check telnet-server is not installed] ************************************************
[WARNING]: Consider using the yum, dnf or zypper module rather than running 'rpm'.  If you
need to use command because yum, dnf or zypper is insufficient you can add 'warn: false' to
this command task or set 'command_warnings=False' in ansible.cfg to get rid of this message.
ok: [TullC-2609-STIG2]
ok: [TullC-2609-STIG1]

TASK [Report telnet removal status] ********************************************************
ok: [TullC-2609-STIG1] => {
    "msg": "telnet-server removed: PASS"
}
ok: [TullC-2609-STIG2] => {
    "msg": "telnet-server removed: PASS"
}

PLAY RECAP *********************************************************************************
TullC-2609-STIG1           : ok=12   changed=0    unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0
TullC-2609-STIG2           : ok=12   changed=0    unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0

(ansible29) [charles@localhost stig-project]$
```

With both droplets provisioned from one playbook, hardened by a single script, and validated as identical, the cloud automation procedure is complete.

FINAL SCRIPT

For submission I combined the three separate playbooks into one single script named 04-final_script.yml, which the project requires ("complete the entire automation process using a single Ansible script"). It runs all three stages in one execution: it provisions the two droplets, registers their live IP addresses into an in-memory inventory with add_host so no manual inventory edit is needed, then hardens and validates both machines. The complete script is below.

04-final_script.yml

```
---
# ============================================================================
# 04-final_script.yml
# Project : Project & Portfolio VII - System Hardening with Ansible & DISA STIGs
# Author  : Charles Tull      Term: 2609
# Purpose : Single-script automation. In one run it:
#   PLAY 1 - provisions two Rocky Linux 8 droplets on DigitalOcean
#   PLAY 2 - applies 27 DISA STIG (RHEL 8) hardening controls
#   PLAY 3 - validates that the controls are in effect on both hosts
# Requires: DO_API_TOKEN exported as an environment variable.
# Run with: ansible-playbook 04-final_script.yml
# ============================================================================

# ######################## PLAY 1 - PROVISION ########################
- name: Provision two DigitalOcean droplets
  hosts: localhost
  connection: local
  gather_facts: false
  vars:
    do_token: "{{ lookup('env', 'DO_API_TOKEN') }}"
    ssh_key_name: "Charles-Tull-2609"
    droplet_region: "nyc1"
    droplet_size: "s-1vcpu-1gb"
    droplet_image: "rockylinux-8-x64"
    droplet_names:
      - "TullC-2609-STIG1"
      - "TullC-2609-STIG2"
```

```
tasks:
  - name: Look up SSH keys on the DO account
    digital_ocean_sshkey_info: { oauth_token: "{{ do_token }}" }
    register: do_keys
  - name: Extract the ID of our named key
    set_fact:
      ssh_key_id: "{{ (do_keys.data | selectattr('name','equalto', ssh_key_name) | list | first).id }}"
  - name: Create the two droplets
    digital_ocean_droplet:
      state: present
      oauth_token: "{{ do_token }}"
      name: "{{ item }}"
      unique_name: true
      region: "{{ droplet_region }}"
      size: "{{ droplet_size }}"
      image: "{{ droplet_image }}"
      ssh_keys: [ "{{ ssh_key_id }}" ]
    loop: "{{ droplet_names }}"
    register: droplets
  # Register each new droplet's live IP into an in-memory group so the later
  # plays can reach them in this same run - no separate inventory needed.
  - name: Add the new droplets to the in-memory inventory
    add_host:
      name: "{{ item.data.droplet.name }}"
      groups: stig_targets
      ansible_host: "{{ item.data.ip_address }}"
      ansible_user: root
      ansible_ssh_private_key_file: ~/.ssh/Charles-Tull-2609
      ansible_python_interpreter: /usr/libexec/platform-python
      ansible_ssh_common_args: '-o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null'
    loop: "{{ droplets.results }}"
    loop_control: { label: "{{ item.item }}" }
  - name: Wait for SSH to come up on each droplet
    wait_for: { host: "{{ item.data.ip_address }}", port: 22, delay: 5, timeout: 300, state: started }
    loop: "{{ droplets.results }}"
    loop_control: { label: "{{ item.item }}" }
```

```
    - name: Show the assigned public IPs

      debug: { msg: "{{ item.data.droplet.name }} -> {{ item.data.ip_address | default('IP pending...')
}}" }

      loop: "{{ droplets.results }}"

      loop_control: { label: "{{ item.item }}" }


# ######################## PLAY 2 - HARDEN (27 STIGs) ########################

- name: Apply DISA STIG hardening to both droplets

  hosts: stig_targets

  become: true

  handlers:

    - name: restart sshd

      service: { name: sshd, state: restarted }

  tasks:

    # ----- GROUP 1: SSH HARDENING -----

    - name: RHEL-08-010550 - Disable SSH root password login

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?PermitRootLogin', line: 'PermitRootLogin
prohibit-password' }

      notify: restart sshd

    - name: RHEL-08-010830 - Disable SSH empty passwords

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?PermitEmptyPasswords', line:
'PermitEmptyPasswords no' }

      notify: restart sshd

    - name: RHEL-08-010200 - Set SSH idle timeout (600s)

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?ClientAliveInterval', line:
'ClientAliveInterval 600' }

      notify: restart sshd

    - name: RHEL-08-010201 - Set SSH ClientAliveCountMax to 1

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?ClientAliveCountMax', line:
'ClientAliveCountMax 1' }

      notify: restart sshd

    - name: RHEL-08-020022 - Disable SSH X11 forwarding

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?X11Forwarding', line: 'X11Forwarding no' }

      notify: restart sshd

    - name: RHEL-08-010522 - Disable SSH PermitUserEnvironment

      lineinfile: { path: /etc/ssh/sshd_config, regexp: '^#?PermitUserEnvironment', line:
'PermitUserEnvironment no' }

      notify: restart sshd

    # ----- GROUP 2: PASSWORD & AUTHENTICATION POLICY -----
```

```
    - name: RHEL-08-020230 - Minimum password length 15

      lineinfile: { path: /etc/security/pwquality.conf, regexp: '^#?minlen', line: 'minlen = 15' }

    - name: RHEL-08-0201xx - Require all 4 character classes

      lineinfile: { path: /etc/security/pwquality.conf, regexp: "^#?{{ item.key }}", line: "{{ item.key }}
= {{ item.value }}" }

      loop:

        - { key: 'ucredit', value: '-1' }

        - { key: 'lcredit', value: '-1' }

        - { key: 'dcredit', value: '-1' }

        - { key: 'ocredit', value: '-1' }

    - name: RHEL-08-020200 - Maximum password age 60 days

      lineinfile: { path: /etc/login.defs, regexp: '^PASS_MAX_DAYS', line: 'PASS_MAX_DAYS 60' }

    - name: RHEL-08-020190 - Minimum password age 1 day

      lineinfile: { path: /etc/login.defs, regexp: '^PASS_MIN_DAYS', line: 'PASS_MIN_DAYS 1' }

    - name: RHEL-08-020221 - Prohibit reuse of last 5 passwords

      lineinfile: { path: /etc/security/pwquality.conf, regexp: '^#?remember', line: 'remember = 5' }

    # ----- GROUP 3: AUDITING (auditd) -----

    - name: RHEL-08-030180 - Ensure auditd is installed

      package: { name: audit, state: present }

    - name: RHEL-08-030181 - Ensure auditd is enabled and running

      service: { name: auditd, state: started, enabled: true }

    - name: RHEL-08-030610 - Restrict audit log permissions

      file: { path: /var/log/audit/audit.log, mode: '0600', state: file }

      ignore_errors: true

    - name: RHEL-08-030550 - Audit privileged (sudo) commands

      lineinfile: { path: /etc/audit/rules.d/stig.rules, line: '-w /usr/bin/sudo -p x -k privileged',
create: true }

    # ----- GROUP 4: KERNEL & NETWORK (sysctl) -----

    - name: RHEL-08-040259 - Disable ICMP redirect acceptance

      sysctl: { name: net.ipv4.conf.all.accept_redirects, value: '0', state: present, sysctl_set: true,
reload: true }

    - name: RHEL-08-040249 - Disable source-routed packets

      sysctl: { name: net.ipv4.conf.all.accept_source_route, value: '0', state: present, sysctl_set: true,
reload: true }

    - name: RHEL-08-040239 - Enable reverse path filtering

      sysctl: { name: net.ipv4.conf.all.rp_filter, value: '1', state: present, sysctl_set: true, reload:
true }

    - name: RHEL-08-040281 - Log martian packets
```

```
      sysctl: { name: net.ipv4.conf.all.log_martians, value: '1', state: present, sysctl_set: true,
reload: true }

    - name: RHEL-08-010375 - Enable kernel ASLR

      sysctl: { name: kernel.randomize_va_space, value: '2', state: present, sysctl_set: true, reload:
true }

    # ----- GROUP 5: FILE PERMISSIONS & SYSTEM INTEGRITY -----

    - name: RHEL-08-010340 - /etc/passwd permissions 0644

      file: { path: /etc/passwd, owner: root, group: root, mode: '0644' }

    - name: RHEL-08-010030 - /etc/shadow permissions 0000

      file: { path: /etc/shadow, owner: root, group: root, mode: '0000' }

    - name: RHEL-08-010350 - /etc/gshadow permissions 0000

      file: { path: /etc/gshadow, owner: root, group: root, mode: '0000' }

    - name: RHEL-08-010060 - Configure login warning banner

      copy:

        dest: /etc/issue

        content: "You are accessing a U.S. Government (USG) Information System (IS) for USG-authorized use
only.\n"

        owner: root

        group: root

        mode: '0644'

    - name: RHEL-08-040170 - Disable Ctrl-Alt-Del burst action

      systemd: { name: ctrl-alt-del.target, masked: true }

    # ----- GROUP 6: INSECURE SERVICES -----

    - name: RHEL-08-040000 - Ensure telnet-server is removed

      package: { name: telnet-server, state: absent }


# ######################## PLAY 3 - VALIDATE ########################

- name: Validate DISA STIG compliance on both droplets

  hosts: stig_targets

  become: true

  gather_facts: false

  tasks:

    - name: Read effective sshd configuration

      command: sshd -T

      register: sshd_config

      changed_when: false

    - name: Report SSH hardening status

      debug:
```

```
        msg: >-
          PermitRootLogin: {{ 'PASS' if 'permitrootlogin prohibit-password' in sshd_config.stdout else
'FAIL' }}
    - name: Check auditd is active
      command: systemctl is-active auditd
      register: auditd_check
      changed_when: false
      failed_when: false
    - name: Report auditd status
      debug: { msg: "auditd running: {{ 'PASS' if auditd_check.stdout == 'active' else 'FAIL' }}" }
    - name: Read /etc/shadow permissions
      stat: { path: /etc/shadow }
      register: shadow_stat
    - name: Report /etc/shadow permission status
      debug: { msg: "/etc/shadow mode 0000: {{ 'PASS' if shadow_stat.stat.mode == '0000' else 'FAIL' }}" }
    - name: Check telnet-server is not installed
      command: rpm -q telnet-server
      register: telnet_check
      changed_when: false
      failed_when: false
    - name: Report telnet removal status
      debug: { msg: "telnet-server removed: {{ 'PASS' if 'not installed' in telnet_check.stdout else
'FAIL' }}" }
```

Final Run - Verifying Everything Together

To verify the whole workflow end to end, I ran the single combined script. Because provisioning is idempotent, it confirmed the existing droplets, waited for SSH, re-applied the hardening, and printed the validation results - all in one run:

```
ansible-playbook 04-final_script.yml
```

The final PLAY RECAP shows all three plays completing with failed=0: localhost (the provisioning play) and both droplets, TullC-2609-STIG1 and TullC-2609-STIG2, with identical results (ok=39, changed=1). The validation checks above the recap all report PASS on both hosts. This single run is the proof that the entire automation - provision, harden, and validate - works together from one script and produces identical outcomes on both machines.

```
ok: [TullC-2609-STIG1] => {
    "msg": "/etc/shadow mode 0000: PASS"
}
ok: [TullC-2609-STIG2] => {
    "msg": "/etc/shadow mode 0000: PASS"
}

TASK [Check telnet-server is not installed] ************************************************
[WARNING]: Consider using the yum, dnf or zypper module rather than running 'rpm'.  If you
need to use command because yum, dnf or zypper is insufficient you can add 'warn: false' to
this command task or set 'command_warnings=False' in ansible.cfg to get rid of this message.
ok: [TullC-2609-STIG2]
ok: [TullC-2609-STIG1]

TASK [Report telnet removal status] ********************************************************
ok: [TullC-2609-STIG1] => {
    "msg": "telnet-server removed: PASS"
}
ok: [TullC-2609-STIG2] => {
    "msg": "telnet-server removed: PASS"
}

PLAY RECAP *********************************************************************************
TullC-2609-STIG1           : ok=39   changed=1    unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0
TullC-2609-STIG2           : ok=39   changed=1    unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0
localhost                  : ok=6    changed=1    unreachable=0    failed=0    skipped=0    re
scued=0    ignored=0

(ansible29) [charles@localhost stig-project]$
```

LESSONS LEARNED

This project went smoothly overall, but I ran into several practical problems along the way. Documenting them here captures both what tripped me up and what I would do differently next time.

Challenges and What I Learned

1. Run commands on the right machine. My first errors came from running Linux commands (dnf) in my Mac terminal, and later from an scp that copied files from the VM back to the VM instead of to my Mac. I learned to always confirm which host the prompt belongs to before running anything - "charles@localhost" is the Rocky VM, while "charlestull@Charless-MacBook-Pro" is the Mac - because a command that is correct on one machine fails or does the wrong thing on the other.

2. A Minimal install really is minimal. The control node was a Rocky Minimal install, so common tools like nano and zip were not present and returned "command not found," and my user was not in the sudoers file because I had not checked "Make this user administrator" during setup. I learned to expect a minimal system to ship almost nothing, to install tools on demand with dnf, and to grant sudo rights up front (or add the user to the wheel group afterward).

3. Pin software versions deliberately. Ansible 2.9 would not run correctly until I pinned it to an exact version and constrained Jinja2 below 3.1, because newer Jinja2 removed functions that Ansible 2.9 depends on. I learned to use a Python virtual environment and pin exact versions so the environment is reproducible and does not break when newer packages are pulled in.

4. Never expose secrets. At one point I pasted my DigitalOcean API token where it could be seen and had to revoke it immediately and generate a new one. I learned to treat access tokens like passwords: load them from an environment variable instead of writing them into the playbook, and rotate them right away if they are ever exposed.

5. Idempotency is the proof of consistency. Because every hardening task enforces a desired state rather than blindly running a command, running the script a second time reported changed=0. That re-run is what actually proves both machines ended up in the same hardened state - it is much stronger evidence than a single run, and I now see idempotency as a feature to design for, not just a nice side effect.

Successes

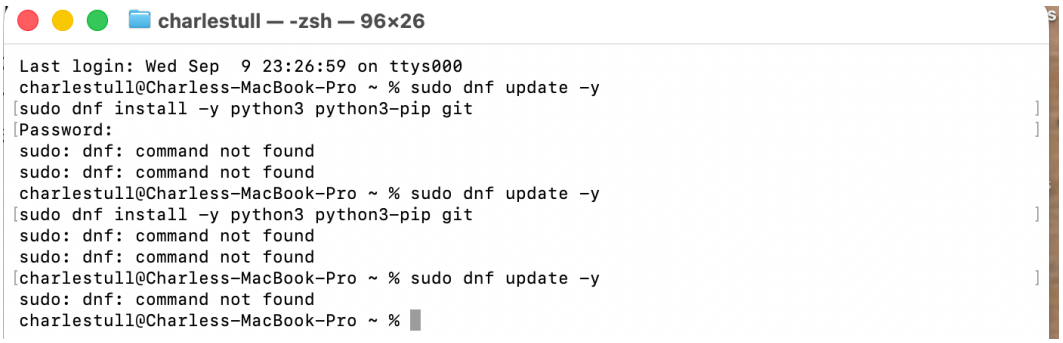
Full end-to-end automation. I combined provisioning, hardening, and validation into a single Ansible script that uses add_host to hand the new droplet IPs from one play to the next, so the whole project runs from one command. The single run provisions both droplets, applies all 27 STIG controls, and validates them - satisfying the "single Ansible script" requirement.

Identical, verified outcomes. The same script hardened both droplets to identical results (ok=39, changed=1, failed=0 on each in the combined run), and a dedicated validation playbook confirmed every checked control reported PASS on both hosts. Proving the two machines are configured identically was the core goal, and the automation demonstrated it clearly.

If I were to extend this project, I would scope the DigitalOcean API token to only the permissions the automation needs (rather than full access), add the remaining STIG controls to cover a full baseline, and set the droplet IP addresses as static so the inventory never has to be updated.

APPENDIX A | DOCUMENT ERRORS & SOLUTIONS

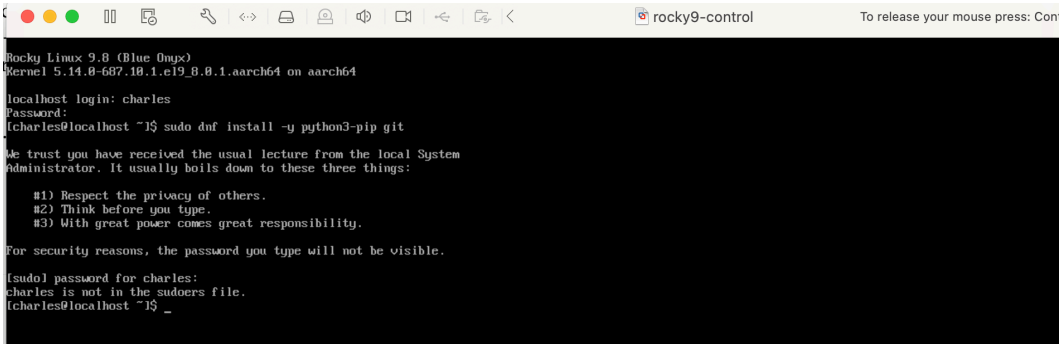
1 - Error Title: sudo: dnf: command not found



Error explanation: I ran the Ansible setup commands (sudo dnf update / sudo dnf install) in my Mac's own terminal by mistake. dnf is the Rocky/RHEL package manager and does not exist on macOS, so every line returned "command not found." The commands were correct - they were simply being run on the wrong operating system.

Solution: I built the Rocky Linux control VM in VMware Fusion and ran the commands inside that Linux VM (and later over SSH from the Mac). Once I was on the Linux box, dnf was available and the packages installed successfully.

2 - Error Title: charles is not in the sudoers file



Error explanation: When I installed Rocky Linux I created the "charles" user but did not check "Make this user administrator," so the account had no sudo privileges. The first sudo command therefore returned "charles is not in the sudoers file. This incident will be reported."

Solution: I switched to the root account with "su -", added my user to the sudo-enabled wheel group with "usermod -aG wheel charles," then logged out and back in so the new group membership applied. Running "sudo whoami" afterward returned "root," confirming the fix.

3 - Error Title: nano / zip: command not found



Error explanation: The control node was a Rocky "Minimal Install," which does not include many common utilities. When I tried to edit a file with nano, and later to archive my playbooks with zip, both returned "command not found" because those packages were not installed.

Solution: I installed each package on demand with "sudo dnf install -y nano" and "sudo dnf install -y zip." Both tools worked immediately afterward.

APPENDIX B | DISA STIGS LIST

STIG #	Risk Category	Description
RHEL-07-010020	(Cat 1) High	cryptographic hash of system files and commands matches vendor values
RHEL-07-010020	(Cat 1) High	Reinstallation of packages necessary for above audit
RHEL-08-010550	(Cat 2) Medium	SSH must not permit direct root login with a password; key-based root access only
RHEL-08-010830	(Cat 2) Medium	SSH must not allow authentication using an empty password
RHEL-08-010200	(Cat 3) Low	SSH must terminate idle sessions after a 10-minute (600 second) period of inactivity
RHEL-08-010201	(Cat 3) Low	SSH must drop the connection after one missed client-alive keepalive
RHEL-08-020022	(Cat 2) Medium	SSH must disable X11 forwarding to reduce remote attack surface
RHEL-08-010522	(Cat 2) Medium	SSH must not permit user environment options that can bypass restrictions
RHEL-08-020230	(Cat 2) Medium	Passwords must be a minimum of 15 characters in length
RHEL-08-020120	(Cat 2) Medium	Passwords must contain at least one uppercase, lowercase, numeric, and special character
RHEL-08-020200	(Cat 2) Medium	Passwords must have a maximum lifetime of 60 days
RHEL-08-020190	(Cat 2) Medium	Passwords must have a minimum lifetime of at least 1 day
RHEL-08-020221	(Cat 2) Medium	The system must prohibit the reuse of the last 5 passwords
RHEL-08-030180	(Cat 2) Medium	The audit package (auditd) must be installed to record security-relevant events

RHEL-08-030181	(Cat 2) Medium	The auditd service must be enabled at boot and actively running
RHEL-08-030610	(Cat 2) Medium	Audit log files must be restricted so they are not readable by unauthorized users
RHEL-08-030550	(Cat 2) Medium	The system must audit all executions of privileged (sudo) commands
RHEL-08-040259	(Cat 2) Medium	The system must not accept IPv4 ICMP redirect messages, preventing route hijacking
RHEL-08-040249	(Cat 2) Medium	The system must not accept IPv4 source-routed packets, defending against spoofing
RHEL-08-040239	(Cat 2) Medium	The system must enable reverse-path filtering to drop packets with spoofed source addresses
RHEL-08-040281	(Cat 3) Low	The system must log packets with impossible (martian) source addresses
RHEL-08-010375	(Cat 2) Medium	The kernel must implement full address space layout randomization (ASLR)
RHEL-08-010340	(Cat 2) Medium	The /etc/passwd file must have permissions set to 0644
RHEL-08-010030	(Cat 1) High	The /etc/shadow file (password hashes) must have permissions set to 0000
RHEL-08-010350	(Cat 2) Medium	The /etc/gshadow file must have permissions set to 0000
RHEL-08-010060	(Cat 2) Medium	The system must display a Standard Mandatory DoD login warning banner before granting access
RHEL-08-040170	(Cat 1) High	The Ctrl-Alt-Delete key sequence must be disabled to prevent an unauthorized console reboot
RHEL-08-040000	(Cat 1) High	The insecure telnet-server package must not be installed (telnet transmits credentials in plaintext)
RHEL-08-040060	(Cat 2) Medium	A single Ansible script applies all controls identically to both hosts for repeatable, continuous enforcement